数据库恢复系统详解

一、恢复系统概述

1.1 恢复算法的重要性

问题场景：假设事务T_i从账户A转50美元到账户B

- 需要两个更新操作：从A减去50，向B加上50
- 如果系统在执行完第一个更新后崩溃，数据库将处于不一致状态

恢复算法的两个组成部分：

1. **正常事务处理期间**：确保有足够信息用于故障恢复
2. **故障发生后**：将数据库恢复到确保原子性、一致性和持久性的状态

1.2 故障分类

故障类型	描述	示例
事务故障	逻辑错误或系统错误导致事务无法完成	死锁、违反完整性约束
系统崩溃	电源故障等导致系统停止	电源中断、操作系统崩溃
磁盘故障	磁盘损坏导致数据丢失	磁头碰撞、磁盘坏道

二、存储结构

2.1 存储类型对比

```
-- 示例：数据访问过程
-- 事务Ti的工作区操作
READ(X)   -- 将数据项X的值赋给局部变量xi
WRITE(X) -- 将局部变量xi的值赋给缓冲区中的数据项X
```

存储层次：

- **易失性存储**：主内存、缓存（不存活于系统崩溃）
- **非易失性存储**：磁盘、磁带、闪存（存活于系统崩溃）
- **稳定存储**：通过多副本实现的理想存储

2.2 稳定存储实现

双重写入策略：

1. 将信息写入第一个物理块
2. 第一个写入成功后，将相同信息写入第二个物理块
3. 仅当第二个写入成功完成后，输出操作才完成

三、日志恢复机制

3.1 日志记录类型

关键日志记录：

- $<T_i$ start$>$：事务T_i开始
- $<T_i, X, V_1, V_2>$：事务T_i更新数据项X（旧值V_1，新值V_2）
- $<T_i$ commit$>$：事务T_i提交
- $<T_i$ abort$>$：事务T_i中止

3.2 立即数据库修改 vs 延迟数据库修改

立即修改示例 ：

```
-- 事务T₀操作序列
BEGIN TRANSACTION T₀;
UPDATE account SET balance = balance - 50 WHERE account_no = 'A';
UPDATE account SET balance = balance + 50 WHERE account_no = 'B';
COMMIT T₀;
```

对应的日志记录 ：

```
<T₀ start>
<T₀, A, 1000, 950>  -- A从1000减少到950
<T₀, B, 2000, 2050> -- B从2000增加到2050
<T₀ commit>
```

3.3 恢复操作

UNDO操作 ：将事务更新的所有数据项恢复为旧值

REDO操作 ：将事务更新的所有数据项设置为新值

恢复判定规则 ：

- 需要UNDO：日志包含 `<Tᵢ start>`但不包含 `<Tᵢ commit>`或 `<Tᵢ abort>`
- 需要REDO：日志包含 `<Tᵢ start>`和 `<Tᵢ commit>`或 `<Tᵢ abort>`

四、检查点技术

4.1 检查点操作

检查点执行步骤 ：

1. 将主内存中的所有日志记录输出到稳定存储
2. 将所有修改的缓冲区块输出到磁盘
3. 写入 `<checkpoint L>`日志记录（L是检查点时活动事务列表）

4.2 检查点恢复优势

恢复范围缩小 ：只需考虑检查点后开始的事务和检查点时活动的事务

示例 （课件第25页图示）：

- T_1：可忽略（更新已在检查点时输出到磁盘）
- T_2和T_3：需要REDO
- T_4：需要UNDO

五、恢复算法详解

5.1 正常操作期间的日志记录

```
# 伪代码：事务执行流程
def transaction_execution(T_i):
    write_log(f"<{T_i} start>")

    for each update operation:
        old_value = read_from_database(X)
        new_value = calculate_new_value(old_value)
        write_log(f"<{T_i}, {X}, {old_value}, {new_value}>")
        write_to_database(X, new_value)

    write_log(f"<{T_i} commit>")
```

5.2 恢复算法步骤

REDO阶段 ：

1. 找到最后一个 <checkpoint L>记录，设置undo-list = L
2. 从检查点记录开始向前扫描
3. 重做所有更新操作
4. 维护undo-list（添加开始事务，移除提交/中止事务）

UNDO阶段 ：

1. 从日志末尾向后扫描
2. 对undo-list中的事务执行UNDO操作
3. 写入补偿日志记录
4. 当找到所有事务的 <Ti start>记录时停止

六、缓冲区管理

6.1 写入策略

No-force策略 ：提交时不需要将更新块写入磁盘

Steal策略 ：允许将未提交事务的更新块写入磁盘

WAL规则（预写日志） ：在数据块输出到磁盘前，所有相关的日志记录必须已输出到稳定存储

6.2 缓冲区输出流程

```
def output_block_to_disk(block_B):
    acquire_exclusive_latch(block_B)  # 获取排他锁
    flush_log_records()               # 强制输出日志记录
    write_block_to_disk(block_B)      # 输出块到磁盘
    release_latch(block_B)            # 释放锁
```

七、例题精讲

例题1：日志分析恢复

题目 ：根据以下日志记录，确定恢复时需要执行的操作：

```
<T₁ start>
<T₁, A, 100, 150>
<T₂ start>
<T₂, B, 200, 250>
<T₁ commit>
<T₃ start>
<T₃, C, 300, 350>
系统崩溃...
```

解答 ：

- T_1：需要REDO（有start和commit记录）
- T_2：需要UNDO（有start但无commit/abort）
- T_3：需要UNDO（有start但无commit/abort）

恢复操作 ：

1. REDO T_1：将A设置为150
2. UNDO T_2：将B恢复为200
3. UNDO T_3：将C恢复为300

例题2：检查点恢复

题目 ：在以下时间点发生系统崩溃，确定恢复范围：

时间点：检查点之前
日志：<T₁ start>, <T₁, A, 10, 20>, <checkpoint L={T₁}>, <T₁ commit>, <T₂ start>, <T₂, B, 30, 40>

解答 ：

- T_1：已提交，需要REDO
- T_2：检查点后开始，需要UNDO（因崩溃时未提交）

例题3：WAL规则应用

题目 ：解释为什么在输出修改的数据块到磁盘前必须先输出日志记录？

解答 ：如果先输出数据块后写日志，系统崩溃时可能无法确定数据块是否包含未提交的更新，从而无法保证原子性。WAL规则确保有足够的日志信息来撤销或重做操作。

八、重点难点总结

8.1 易错概念

1. **日志记录顺序** ：日志记录必须先于实际数据写入稳定存储
2. **检查点意义** ：不是将数据立即写入磁盘，而是记录恢复起点
3. **补偿日志记录** ：UNDO操作本身也需要记录日志，防止重复撤销

8.2 真题关联分析

根据提供的期末考试真题，恢复系统相关考点包括：

- 事务的ACID特性（简答题第2题）
- 系统故障恢复策略（简答题第4题）
- 并发控制与锁机制（选择题第10题）

这些知识点都在本章有详细讲解，需要重点掌握。

学习建议 ：恢复系统是数据库的核心机制，建议通过绘制日志序列图、模拟不同故障场景的恢复过程来加深理解。多做练习题，特别是涉及复杂事务序列的恢复分析题目。